

CLASE: Introducción a la Programación

La programación es el arte de escribir instrucciones para que una computadora realice tareas específicas. Python, uno de los lenguajes más populares, destaca por su sintaxis sencilla y su versatilidad en el manejo de datos.

En esta clase, exploraremos los conceptos fundamentales que necesitas para comenzar en la programación, desde estructuras básicas hasta herramientas avanzadas para el análisis de datos.

Temas que abordaremos:

- **Plataformas de Colaboración para Ciencia de Datos:** Trabajaremos con DeepNote, esta aplicación web proporciona un entorno en línea donde los usuarios pueden crear, ejecutar y compartir notebooks de Jupyter de manera colaborativa, sin distraerse con la gestión de archivos o la configuración del entorno.
- **Introducción a Python:** Aprenderemos por qué Python es tan popular y cómo iniciar con este lenguaje.
- **Estructuras de control de flujo:** Veremos cómo controlar la ejecución de un programa con condicionales y bucles.
- **Estructuras de datos en Python:** Exploraremos los principales contenedores de datos como listas, tuplas, conjuntos y diccionarios, fundamentales para organizar información de manera eficiente.

Plataformas de Colaboración para Ciencia de Datos

¿Qué es DeepNote?

Deepnote es una plataforma de colaboración para la ciencia de datos que permite a los equipos trabajar juntos de manera eficiente. Es compatible con Jupyter Notebooks y ofrece funciones de integración y visualización que mejoran el flujo de trabajo de los científicos de datos. Además, al operar completamente en la nube, elimina la necesidad de instalaciones locales, facilitando el acceso desde cualquier lugar.

Se puede usar Jupyter Notebooks de manera muy parecida a Google Colaboratory. Con algunas ventajas, por ejemplo, los bloques de markdown son más potentes y están mejor organizados que los de Colab, incluso soportan HTML. Conforme vas escribiendo, se crea una tabla de contenidos acorde a los títulos que coloques, lo que permite una fácil navegación por el notebook.

Características de Deepnote

Antes de seguir, dejo un disclaimer, la plataforma se actualiza periódicamente, por lo que, es posible que lo que explique hoy no se vea igual, por suerte las actualizaciones suelen ser para mejor.

- Si bien tiene versiones premium, para los fines educativos de esta materia, la versión gratuita nos funciona de maravilla, así que lo primero, con la cuenta institucional @ifts24.edu.ar nos suscribimos a la plataforma.
- Trabaja por proyectos, que son como entornos virtuales en los que puedes crear varios notebooks y tener varios tipos de archivos.
- Tiene un montón de atajos de teclado y una paleta de comandos que harán que tu trabajo sea

más sencillo y a mayor velocidad que otras plataformas.

- Es posible colaborar en tiempo real con otras personas de la misma manera que lo haces en Google Docs, una ventaja competitiva frente a Colab.
- Puedes trabajar con distintas versiones y crear tu propio entorno personalizado.
- Optimización de las visualizaciones para los proyectos de Data Science: visualización estructurada de csv, previsualización enriquecida de los DataFrames de Pandas, función para crear gráficas sin código a partir de datasets, explorador de variables globales, las puedes encontrar debajo de la tabla de contenidos, por defecto verán el shape o el type.
- Tiene muchas integraciones que pueden ser compartidas si estás trabajando en equipo, conectando con MongoDB, PostgreSQL, Amazon S3, Google Drive, Spark, entre otros.
- Te brinda la posibilidad de conectar con GitHub en forma directa, además, tiene integrada su propia terminal de Linux con la que se pueden ejecutar todos los comandos de Git.
- Sistema de versionado para controlar todos los cambios hechos por cada persona y poder regresar a ellos.
- Permite calendarizar la ejecución de tus notebooks para que se ejecuten diaria o semanalmente sin que sea necesario que estés presente.
- Si pasas el mouse por encima de una función o método, verás su documentación, de la misma forma que si usaras `help(function)`. Y si aplastas Ctrl + click a una variable, te llevará al bloque de código en la que la declaraste.
- Al publicar tus notebooks como proyectos aparecerán a modo de portafolio.
- Para ver todas las actualizaciones pueden ingresar a <https://deepnote.com/docs/getting-started>

Atajos de teclado en Deepnote

Los shortcuts de Deepnote hacen que el flujo de trabajo sea mucho más dinámico. Si los aprenden notarán una gran diferencia al momento de trabajar, la herramienta es como cualquier otra, se aprende a usar, usando, así que, como siguiente medida lo que tienen que hacer es: abrir la aplicación, registrarse, crear un notebook y empezar a usarlo.

Shortcut	Función
Ctrl + Enter	Ejecuta el bloque de código.
Shift + Enter	Ejecuta el bloque de código y pasa al siguiente.
Ctrl + Shift + .	Detiene la ejecución.
Ctrl + Shift + M	Transforma un bloque de código a uno de markdown.
Ctrl + Shift + Y	Transforma un bloque de markdown a uno de código.
Ctrl + J	Crea un nuevo bloque debajo.
Ctrl + K	Crea un nuevo bloque encima.
Ctrl + Shift + Del	Elimina un bloque.
Alt + Shift + ↑	Mueve un bloque hacia arriba.
Alt + Shift + ↓	Mueve un bloque hacia abajo.
Ctrl + Shift + D	Duplica un bloque.
Ctrl + Shift + H	Oculto/muestra el output de un bloque de código.
Ctrl + Alt + H	Oculto/muestra el código de un bloque.
Ctrl + Alt + C	Añade un comentario.
Ctrl + P	Abre la paleta de comandos.

Algunos comandos especiales dentro de un bloque de código, que son una maravilla:

- Si estás sobre una variable y presionas **Ctrl + D** varias veces, seleccionarás todas veces que llamaste a esa variable y podrás cambiarle su nombre.
- De manera parecida al shortcut anterior, si presionas **Alt + click** en varias partes del código, podrás escribir/modificar simultáneas líneas.
- Con **Alt + ↑ o ↓** puedes mover una línea de código en específico para arriba o abajo.
- Puedes formatear el bloque de código abriendo la paleta de comandos y buscando: *“Format block”*.

Publicar notebooks como portafolio

Si te posicionas en “Share”, en la parte superior derecha del notebook, y luego en “Publishing”, podrás compartir tu proyecto como si fuera un artículo en tu perfil con estilo de portafolio. Otra forma de publicación es como “publish it as unlisted”.

Trabajo colaborativo

En tu dashboard tienes la opción de crear un equipo. Cuando lo hagas, las personas invitadas podrán acceder a todos los proyectos que se creen. Esto hace que trabajar en equipo sea diferente a cuando compartes tus proyectos personales a colaboradores ocasionales. Todo el equipo podrá tener las mismas integraciones. También podrás administrar los permisos para cada uno de los miembros.

Recuerden que estamos trabajando con el plan gratuito, este plan permite formar un equipo de hasta 3 integrantes con un límite de 3 proyectos. Igual en un proyecto pueden tener un montón de notebooks.

Subir imágenes con markdown

Para subir imágenes a los bloques de markdown, lo único que debes hacer es copiar la imagen y pegarla en un bloque de markdown, a continuación te aparecerá un código como este en un nuevo bloque:

```
![image-name.png](ruta-de-la-imagen.png)
```

Tengan en cuenta si suben muchas imágenes, la parte de los archivos se llenará y quedará muy desordenado. Lo que pueden hacer es crear una nueva carpeta llamada “images” y luego arrastrar todas las imágenes ahí. Luego para llamarlas solo tienen que agregar **images/** al inicio de la ruta de cada imagen como se muestra a continuación:

```
![image-name.png](images/ruta-de-la-imagen.png)
```

Al en Deepnote

Deepnote ofrece ayuda contextual con IA para todos tus proyectos de datos. Deepnote comprende a fondo sus proyectos, almacenes de datos y metadatos. Ofrece asistencia de IA precisa y auditable para tus trabajos con datos. La estructura de cuaderno garantiza que cada sugerencia de IA se convierta en resultados transparentes y reproducibles, lo que permite a los equipos de datos confiar en la IA.

Pueden acelerar el flujo de trabajo y eliminar las tareas tediosas y repetitivas con las sugerencias de código contextuales de la IA de Deepnote. Recorra rápidamente las líneas de código y complete las

sugerencias de funciones, estos son algunos ejemplos de cómo puedes ayudarte con ella:

- Explicar el código: No dejes que fragmentos complejos de código te confundan más con explicaciones concisas y fáciles de entender
- Editar código: Simplemente pídele a Deepnote AI y observa cómo edita cualquier código existente.
- Utilizar código de depuración: La IA de Deepnote identifica el problema y ofrece una solución inmediata con un solo clic, para que no te quedes atascado en tu proceso de trabajo.

Dark mode en Deepnote

Deepnote si tiene modo oscuro, para aplicarlo es tan simple como ir a la ruedita de configuración o “Settings & members”, en la sección de apariencia cambiar en Interface theme *Light* por **Dark** y listo!

Algunos comandos de Pandas y Seaborn

Código	► Función
Pandas	
<code>pd.read_csv('ruta')</code>	-> Lee el DataFrame y lo puedes almacenar en una variable {df}
<code>df.dtypes</code>	-> Los tipos de todas las columnas
<code>df.describe()</code>	-> Conjunto completo de estadísticos descriptivos (fundamentales)
<code>df.groupby('categoria').count()</code>	-> Agrupa los datos por la variable categórica y los cuenta
<code>df['categoria'].mean()</code>	-> Saca la media de todos los datos de esa categoría
<code>df['category'].plot.hist(bins=20)</code>	-> Histograma de frecuencia con intervalos de valores (bins)
Seaborn	
<code>sns.scatterplot(data=dataset, x='x_column', y='y_column')</code>	-> Grafica un scatterplot (diagrama de dispersión). Si añades <code>hue='categoria'</code> , aparece en colores clasificados.
<code>sns.jointplot(data=dataset, x='x_column', y='y_column')</code>	-> Grafica un scatterplot y sus distribuciones. Si añades <code>hue='categoria'</code> , aparece en colores clasificados.
<code>sns.boxplot(data=iris, x='species', y='sepal_length')</code>	-> Grafica los cuartiles.
<code>sns.barplot(data=iris, x='species', y='sepal_length')</code>	-> Grafica un diagrama de barras.

Introducción a la Programación en Python

¿Qué es Python?

Python es un lenguaje de programación de alto nivel, interpretado y orientado a objetos. Fue creado a

finales de los 80 por Guido van Rossum, y desde entonces se ha convertido en uno de los lenguajes de programación más populares en el mundo.

Variables y tipos de datos

En Python, las variables se utilizan para almacenar valores. Para asignar un valor a una variable, simplemente escribe el nombre de la variable seguido del operador de asignación (=) y el valor que deseas asignar.

Por ejemplo:

```
edad = 27
nombre = "Juan"
nombre
'2'
```

- En este ejemplo, hemos creado dos variables: `edad` y `nombre`. La variable `edad` contiene un valor entero (`27`), mientras que la variable `nombre` contiene una cadena de caracteres (`"Juan"`).

Python tiene varios tipos de datos, entre ellos:

- **Enteros (int)**: números enteros, por ejemplo: `27`
- **Flotantes (float)**: números con decimales, por ejemplo: `3.1416`
- **Cadenas de caracteres (str)**: secuencias de caracteres, por ejemplo: `"Juan"`
- **Booleanos (bool)**: valores de verdad `True` o `False`

Operadores aritméticos

Python tiene varios operadores aritméticos que puedes utilizar para realizar cálculos matemáticos. Algunos de los operadores más comunes son:

- **Suma (+)**: se utiliza para sumar dos valores.
- **Resta (-)**: se utiliza para restar dos valores.
- **Multiplicación (*)**: se utiliza para multiplicar dos valores.
- **División (/)**: se utiliza para dividir dos valores.
- **Módulo (%)**: se utiliza para obtener el resto de una división.

Por ejemplo:

```
a = 5
b = 2

suma = a + b # suma es igual a 7
resta = a - b # resta es igual a 3
multiplicacion = a * b # multiplicacion es igual a 10
exponente = a ** b # multiplicacion es igual a 25
division = a / b # division es igual a 2.5
modulo = a % b # modulo es igual a 1
```

```
a = 3
b = 26
division = a / b
división
```

0.11538461538461539

Estructuras de control de flujo

Las estructuras de control de flujo se utilizan en Python para controlar el flujo de ejecución de un programa. Algunas de las estructuras de control de flujo más comunes son:

- **Condicionales:** se utilizan para ejecutar un bloque de código si se cumple una condición determinada. En Python, la estructura de control de flujo condicional más común es el `if`.
- **Bucles:** se utilizan para repetir un bloque de código varias veces. En Python, los bucles más comunes son el `for` y el `while`.

Por ejemplo:

```
#Ejemplo de condicional if
edad = 13

if edad >= 18:
    print("Eres mayor de edad")
else:
    print("Eres menor de edad")
texto1 = "Eres mayor de edad"
texto2 = "Eres menor de edad"
edad = 17
nombre = 'Jose'
if edad <= 18 and nombre == 'Juan':
    print(texto2)
elif edad <= 18:
    print(texto2, 'y no podes entrar')
else:
    print(texto1)
Eres menor de edad y no podes entrar

#Ejemplo de bucle for
numeros = [1, 2, 3, 4, 5, 19]
print(numeros)
[1, 2, 3, 4, 5, 19]
for numero in numeros:
    print(numero)
1
2
3
4
5
19

#Ejemplo de bucle while
numero = 1
while numero <= 5:
    print(numero)
    numero += 1
1
2
3
```

4

5

numero

▶ 6

▶ `print(numero)`

Funciones

Las funciones son bloques de código que se pueden llamar varias veces desde diferentes partes de un programa. Las funciones se definen utilizando la palabra clave ``def``, seguida del nombre de la función y los parámetros que recibe la función (si los tiene). Dentro de la función, se puede utilizar la palabra clave ``return`` para devolver un valor. Por ejemplo:

▶ `def calcular_suma(a, b):` `suma = a + b` `return suma`▶ `calcular_suma(edad,7)`

▶ 24

▶ suma

▶ 7

`resultado = calcular_suma(5,7)`

▶ resultado

▶ 12

Estructuras de datos en Python

Python es un lenguaje de programación que soporta varios tipos de estructuras de datos, cada una de las cuales tiene sus propias características y funcionalidades. A continuación, describiremos las principales estructuras de datos que se utilizan en Python.

Listas

Las listas son una de las estructuras de datos más comunes en Python. Se utilizan para almacenar una colección ordenada de elementos. Los elementos pueden ser de cualquier tipo, como números, cadenas de texto, objetos, etc. Para crear una lista en Python, utilizamos corchetes y separamos los elementos con comas. Por ejemplo, la siguiente línea de código crea una lista con tres elementos:

▶ `mi_lista = [1, 2, 3]`

Podemos acceder a los elementos de una lista utilizando índices. Los índices comienzan en cero, lo que significa que el primer elemento de la lista tiene un índice de 0. Por ejemplo, para acceder al primer elemento de la lista anterior, podemos utilizar el siguiente código:

▶ `primer_elemento = mi_lista[0]`

También podemos modificar elementos de una lista utilizando índices. Por ejemplo, para modificar el

segundo elemento de la lista anterior, podemos utilizar el siguiente código:

```
mi_lista[1] = 4
```

Tuplas

Las tuplas son similares a las listas, pero son inmutables, lo que significa que no se pueden modificar después de su creación. Las tuplas se utilizan a menudo para almacenar datos que no deben cambiar, como coordenadas o valores constantes.

Para crear una tupla en Python, utilizamos paréntesis y separamos los elementos con comas. Por ejemplo, la siguiente línea de código crea una tupla con tres elementos:

```
mi_tupla = (1, 2, 3)
```

Podemos acceder a los elementos de una tupla utilizando índices, al igual que con las listas.

Conjuntos

Los conjuntos son una estructura de datos que se utilizan para almacenar elementos únicos. Los conjuntos no tienen un orden específico, y los elementos no se pueden acceder mediante índices.

Para crear un conjunto en Python, utilizamos llaves y separamos los elementos con comas. Por ejemplo, la siguiente línea de código crea un conjunto con tres elementos:

```
mi_conjunto = {1, 2, 3}
```

Podemos agregar elementos a un conjunto utilizando el método `add`, y podemos eliminar elementos utilizando el método `remove`.

Diccionarios

Los diccionarios son una estructura de datos que se utilizan para almacenar pares clave-valor. Los diccionarios se utilizan a menudo para almacenar información que se puede acceder mediante una clave, como un diccionario real.

Para crear un diccionario en Python, utilizamos llaves y separamos las claves y los valores con dos puntos. Cada par clave-valor se separa con comas. Por ejemplo, la siguiente línea de código crea un diccionario con tres pares clave-valor:

```
mi_diccionario = {'clave1': 'valor1', 'clave2': 'valor2'}
```

Podemos acceder a los valores de un diccionario utilizando las claves. Por ejemplo, para obtener el valor asociado a la clave `'clave1'`, podemos utilizar el siguiente código:

```
valor = mi_diccionario['clave1']
```

También podemos modificar los valores de un diccionario utilizando las claves. Por ejemplo, para cambiar el valor asociado a la clave `'clave2'`, podemos utilizar el siguiente código:

```
mi_diccionario['clave2'] = 'nuevo_valor'
```


Estructuras de datos en Pandas

Pandas es una librería de Python utilizada para el análisis de datos. Pandas utiliza dos estructuras de datos principales: las Series y los DataFrames.

Series

Una Serie es una estructura de datos unidimensional que se utiliza para almacenar datos de un solo tipo. Las Series se parecen a las listas o los arrays de NumPy, pero tienen etiquetas de índice que se utilizan para identificar cada elemento de la serie.

Para crear una Serie en Pandas, podemos utilizar la función `pd.Series()`. Por ejemplo, la siguiente línea de código crea una Serie con tres elementos:

```
import pandas as pd

mi_serie = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
```

Podemos acceder a los elementos de una Serie utilizando las etiquetas de índice, al igual que con los diccionarios. Por ejemplo, para obtener el valor asociado a la etiqueta 'a', podemos utilizar el siguiente código:

```
valor = mi_serie['a']
```

También podemos modificar los valores de una Serie utilizando las etiquetas de índice. Por ejemplo, para cambiar el valor asociado a la etiqueta 'b', podemos utilizar el siguiente código:

```
mi_serie['b'] = 4
```

DataFrames

Un DataFrame es una estructura de datos bidimensional que se utiliza para almacenar datos tabulares. Los DataFrames tienen etiquetas de índice tanto para las filas como para las columnas, y cada columna puede contener datos de un tipo diferente.

Para crear un DataFrame en Pandas, podemos utilizar la función `pd.DataFrame()`. Por ejemplo, la siguiente línea de código crea un DataFrame con dos columnas:

```
mi_dataframe = pd.DataFrame({'columna1': [1, 2, 3], 'columna2': ['a', 'b', 'c']}, index=['fila1', 'fila2', 'fila3'])
```

Podemos acceder a los elementos de un DataFrame utilizando las etiquetas de índice para las filas y las columnas. Por ejemplo, para obtener el valor en la fila 'fila2' y la columna 'columna1', podemos utilizar el siguiente código:

```
valor = mi_dataframe.loc['fila2', 'columna1']
```

También podemos modificar los valores de un DataFrame utilizando las etiquetas de índice. Por ejemplo, para cambiar el valor en la fila 'fila3' y la columna 'columna2', podemos utilizar el siguiente código:

```
mi_dataframe.loc['fila3', 'columna2'] = 'nuevo_valor'
```

Otro Ejemplo

A continuación tienen otro ejemplo para que puedan jugar un poco con el código y experimentar que sucede

```
import pandas as pd
```

```
# Creamos una Serie
```

```
mi_serie = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
```

```
# Accedemos a los elementos de la Serie
```

```
print(mi_serie['a']) # Imprime: 1
```

```
# Modificamos un elemento de la Serie
```

```
mi_serie['b'] = 4
```

```
# Creamos un DataFrame
```

```
mi_dataframe = pd.DataFrame({'columna1': [1, 2, 3], 'columna2': ['a', 'b', 'c']}, index=['fila1', 'fila2', 'fila3'])
```

```
# Accedemos a los elementos del DataFrame
```

```
print(mi_dataframe.loc['fila2', 'columna1']) # Imprime: 2
```

```
# Modificamos un elemento del DataFrame
```

```
mi_dataframe.loc['fila3', 'columna2'] = 'nuevo_valor'
```

```
# Agregamos una nueva columna al DataFrame
```

```
mi_dataframe['columna3'] = [True, False, True]
```

```
# Eliminamos una columna del DataFrame
```

```
mi_dataframe = mi_dataframe.drop('columna2', axis=1)
```

```
# Filtramos las filas del DataFrame
```

```
mi_dataframe_filtrado = mi_dataframe[mi_dataframe['columna1'] > 1]
```

```
# Agrupamos los datos del DataFrame
```

```
mi_dataframe_agrupado = mi_dataframe.groupby('columna3').sum()
```

```
1  
2
```

Bibliotecas para Data Science en Python

Python cuenta con muchas bibliotecas que facilitan la programación en el ámbito de la ciencia de datos. Algunas de las bibliotecas más comunes son:

- **NumPy**: se utiliza para trabajar con arreglos numéricos y matrices. <https://numpy.org/doc/>
- **Pandas**: se utiliza para trabajar con datos estructurados, como tablas o bases de datos. <https://pandas.pydata.org/docs/>
- **Matplotlib**: se utiliza para crear gráficas y visualizaciones.

- <https://matplotlib.org/stable/contents.html>
- Scikit-learn: se utiliza para crear modelos de aprendizaje automático. <https://scikit-learn.org/stable/documentation.html>
- TensorFlow: se utiliza para crear modelos de aprendizaje profundo. https://www.tensorflow.org/api_docs

Para utilizar una biblioteca en Python, primero debes importarla en tu programa utilizando la palabra clave ``import``.

Por ejemplo:

```
➤ import numpy as np
arr = np.array([1, 2, 3, 4, 10])
media = np.mean(arr) # media es igual a 4.0

➤ np.median(arr)
➤ 3.0

➤ print(media)
➤ 4.0
```

Ejemplo práctico

Para ilustrar cómo se pueden utilizar estos conceptos en un ejemplo práctico, vamos a crear un programa que calcule el índice de masa corporal (IMC) de una persona. El IMC es una medida que se utiliza para determinar si una persona tiene un peso saludable en relación con su altura.

Para calcular el IMC, necesitamos la altura (en metros) y el peso (en kilogramos) de la persona. Utilizaremos la fórmula:

IMC = peso / (altura ** 2)

Para comenzar, vamos a pedirle al usuario que ingrese su altura y su peso:

```
altura = float(input("Ingrese su altura (en metros): "))
peso = float(input("Ingrese su peso (en kilogramos): "))
```

```
➤ Ingrese su altura (en metros): 1.8
Ingrese su peso (en kilogramos): 80
```

Después, podemos calcular el IMC utilizando la fórmula:

```
➤ imc = peso / (altura ** 2)
```

Finalmente, podemos imprimir el resultado utilizando la función `print`:

```
➤ print("Tu IMC es:", imc)
```

En conjunto, el programa completo se vería así:

```
➤ altura = float(input("Ingrese su altura (en metros): "))
  peso = float(input("Ingrese su peso (en kilogramos): "))

  imc = peso / (altura ** 2)

  print("Tu IMC es:", imc)
```

- ▶ Ingrese su altura (en metros): 1.70
Ingrese su peso (en kilogramos): 100
Tu IMC es: 34.602076124567475

Con este programa, podemos calcular el IMC de cualquier persona que ingrese su altura y su peso. Este es solo un ejemplo de cómo se pueden utilizar los conceptos de programación en Python para crear soluciones prácticas en el ámbito de la ciencia de datos.

Funciones lambda

Las funciones lambda, también conocidas como funciones anónimas, son funciones pequeñas y simples que no tienen un nombre definido y se utilizan para expresiones o situaciones que no requieren una función completa. Las funciones lambda se definen utilizando la palabra clave `lambda`, seguida de los parámetros de entrada de la función y la expresión que se debe evaluar.

Por ejemplo, la siguiente función lambda toma un argumento `x` y devuelve el cuadrado de `x`:

```
cuadrado = lambda x: x ** 2
```

En este ejemplo, hemos definido una función lambda llamada `cuadrado` que toma un argumento `x` y devuelve el cuadrado de `x`.

- ▶ `cuadrado = lambda x: x ** 2`

Las funciones lambda se pueden utilizar en lugar de las funciones normales en situaciones en las que se requiere una función pequeña y sencilla. Por ejemplo, las funciones lambda se pueden utilizar como argumentos de otras funciones o en expresiones que requieren funciones.

Por ejemplo, la siguiente función `map` utiliza una función lambda para aplicar la función `cuadrado` a cada elemento de la lista `numeros`:

- ▶

```
numeros = [1, 2, 3, 4, 5]  
cuadrados = list(map(lambda x: x ** 2, numeros))  
cuadrados  
[1, 4, 9, 16, 25]
```

En este ejemplo, hemos utilizado la función `map` para aplicar la función lambda `lambda x: x ** 2` a cada elemento de la lista `numeros`. Después, hemos convertido los resultados en una lista utilizando la función `list`.

Es importante mencionar que las funciones lambda solo se utilizan en situaciones en las que se requiere una función pequeña y sencilla. Para funciones más complejas, es recomendable utilizar funciones normales para una mejor legibilidad y mantenibilidad del código.

TIPS de la Profe

Python es increíblemente versátil, ideal para la ciencia de datos, el desarrollo web y más. Estas son algunas de las principales bibliotecas de diferentes dominios:

Manipulación de datos

- **Pandas**: Imprescindible para el análisis de datos.
- **NumPy**: Ideal para matrices y matrices.
- **Vaex**: Maneja grandes conjuntos de datos de manera eficiente.
- **Polars**: Biblioteca rápida de DataFrame en Rust.

Visualización de datos

- **Matplotlib**: Trazado clásico.
- **Seaborn**: Hermosos gráficos estadísticos.
- **Plotly**: Gráficos interactivos.
- **Bokeh**: Parcelas aptas para la web.

Análisis estadístico

- **SciPy**: Para la computación científica.
- **Statsmodels**: Clave para los modelos estadísticos.

Aprendizaje automático

- **Scikit-learn**: Herramientas de ML sencillas.
- **TensorFlow**: Plataforma integral de ML.
- **PyTorch**: Aprendizaje profundo flexible.
- **XGBoost**: Impulso optimizado.

PNL

- **NLTK**: Conjunto completo de herramientas de PNL.
- **SpaCy**: PNL rápido y robusto.
- **Gensim**: Modelado de temas.

Operaciones de base de datos

- **Dask**: Computación paralela.
- **PySpark**: Procesamiento de big data.
- **Koalas**: Parecido a los pandas en Spark.

Series temporales

- **Statsmodels**: Para el análisis de series temporales.
- **Prophet**: Predicción fácil.

Web Scraping

- **Beautiful Soup**: Raspado simple.
- **Scrapy**: Potente rastreo web.
- **Selenium**: Automatización de navegadores.

Explora estas bibliotecas para desbloquear el potencial de Python. ¡**Feliz codificación!**

Fuente **#RavenaO**

¡Ahora te toca a vos!

Antes de empezar con el trabajo práctico, te desafío a poner en práctica lo que aprendimos para los que no conocían y repasamos para los que ya tenían conocimientos. Así que, pongo a disposición una serie de ejercicios iniciales, organizados por temas para que vayan calentando motores.

1. Sintaxis y variables

Objetivo: entender asignación, tipos de datos y print()

- Crear un programa que imprima tu nombre.
- Asignar valores a variables y mostrarlos por pantalla.
- Calcular el área de un rectángulo con base y altura dadas.

2. Operadores y entrada de datos

Objetivo: practicar operaciones matemáticas y input()

- Ingresar dos números y mostrar la suma, resta, multiplicación y división.
- Calcular el promedio de tres notas ingresadas por el usuario.
- Convertir grados Celsius a Fahrenheit.

3. Condicionales (if, else, elif)

Objetivo: controlar el flujo del programa según condiciones

- Determinar si un número es positivo, negativo o cero.
- Verificar si una persona es mayor de edad (edad > 18).
- Decidir si un año es bisiesto.

4. Bucles (for, while)

Objetivo: repetir instrucciones y controlar ciclos

- Imprimir los números del 1 al 10.
- Mostrar la tabla de multiplicar de un número ingresado.
- Sumar todos los números pares entre 1 y 100.

5. Listas y operaciones básicas

Objetivo: entender estructuras de datos simples

- Crear una lista con cinco frutas e imprimir cada una.
- Pedir cinco números al usuario y guardarlos en una lista.
- Calcular el promedio de una lista de números.

6. Funciones

Objetivo: modularizar el código y reutilizar lógica

- Crear una función que calcule el área de un círculo.
- Crear una función que determine si un número es primo.
- Crear una función que reciba una lista y devuelva el mayor número.

Extras opcionales para motivar:

- Adivina el número: juego simple con while, random y if.
- Calculadora básica: suma, resta, multiplicación y división con menú.
- Generador de contraseñas simples.

